

LAB 1 – REST API

Technical Requirements & Design Standards (PRN232)

Assignment Requirement

Develop an **ASP.NET Core RESTful API** using a **3-layer architecture** for a Learning Management System (LMS). The database must contain at least the following tables:

- *Semester(SemesterId int, SemesterName nvarchar(100), StartDate datetime, EndDate datetime)*
- *Course(CourseId int, CourseName nvarchar(100), SemesterId int)*
- *Subject(SubjectId int, SubjectCode varchar(20), SubjectName nvarchar(100), Credit int)*
- *Student(StudentId int, FullName nvarchar(100), Email varchar(100), DateOfBirth datetime)*
- *Enrollment(EnrollmentId int, StudentId int, CourseId int, EnrollmentDate datetime, Status varchar(20))*

Students may add additional tables if needed. Generate data with at least 5 semesters, 50 students, 10 subjects, 20 courses, 500 enrollments.

1. Architecture & Project Structure

Apply a **3-layer architecture**:

- API Layer (Controllers)
- Service Layer (Business Logic)
- Repository Layer (Data Access)

Requirements

- Controllers must not contain business logic.
- Repositories must not contain business logic.
- Clear separation of responsibilities between layers.

Project Naming Convention

PRN232.[ProjectName].API

PRN232.[ProjectName].Services
PRN232.[ProjectName].Repositories

2. Data Model Specification

The project must use 4 model types:

- **Entity Model** → Database mapping
- **Business Model** → Business processing
- **Request Model** → Client input
- **Response Model** → API output

Requirements

- Do not return Entity Models directly in API responses.
- Do not use Request/Response Models in Repository Layer.

3. RESTful API Design

- APIs must follow RESTful principles.
- Use resource-based endpoints.
- Use plural nouns in URLs.

References

- [REST API Naming Guide](#)
- [Microsoft REST API Guidelines](#)

Correct Examples

/api/students
/api/students/{id}
/api/enrollments/{id}

Incorrect Examples

/api/getStudents
/api/createEnrollment

4. GET Resource by ID

- Return complete related data for the resource.
- Avoid circular references and infinite recursion.
- Return HTTP 404 if the resource does not exist.

Example:

/api/students/1
/api/enrollments/10

5. GET Collection Resource (List API)

All list APIs must support the following capabilities:

- **Searching:** filter data by keyword or conditions.
- **Sorting:** sort results by one or multiple fields in ascending/descending order.
- **Paging:** return data in pages using page number and page size.
- **Selection:** allow clients to request specific fields only.
- **Expansion:** allow inclusion of related entities/resources in the response.

GET /students?search=nguyen

GET /students?sort=fullName,-dateOfBirth

GET /students?page=2&size=10

GET /students?fields=studentId,fullName,email

GET /enrollments?expand=student,course

GET
enrollments?search=active&sort=-enrollDate&page=1&size=20&fields=enrollmentId,status&expand=student,course

Pagination Metadata

```
"pagination": {  
  "page": 1,  
  "pageSize": 10,  
  "totalItems": 100,  
  "totalPages": 10  
}
```

6. Response Format & HTTP Status Codes

All APIs must return a consistent response format.

Example

```
{  
  "success": true,  
  "message": "Request processed successfully",  
  "data": {},  
  "errors": null  
}
```

HTTP Status Codes

- 200 — Success
- 201 — Created
- 400 — Bad Request
- 404 — Not Found
- 500 — Internal Server Error

7. Docker Deployment Requirements

- Database must run using Docker Desktop.
- API must run inside Docker containers.
- The project must include:
 - Dockerfile
 - docker-compose.yml

Students must demonstrate that both API and Database run successfully using Docker Compose.

8. Swagger / OpenAPI Documentation

Swagger/OpenAPI integration is required.

Swagger must support:

- Endpoint listing
- API testing

- Request/response documentation
- HTTP status code documentation

References

- [Swagger OpenAPI Specification](#)
- [ASP.NET Core Swagger Documentation](#)

9. Evaluation Checklist

- Correct 3-layer architecture
- Correct use of 4 model types
- RESTful API design
- Search, filter, sort, paging support
- Pagination metadata included
- Consistent response format
- Proper HTTP status codes
- Docker deployment completed
- Swagger/OpenAPI integrated

10. Out of Scope

The following features are NOT required for this lab:

- Authentication / Authorization
- JWT Security
- Advanced Validation
- Global Exception Handling
- Unit Testing / Integration Testing